CJAVA

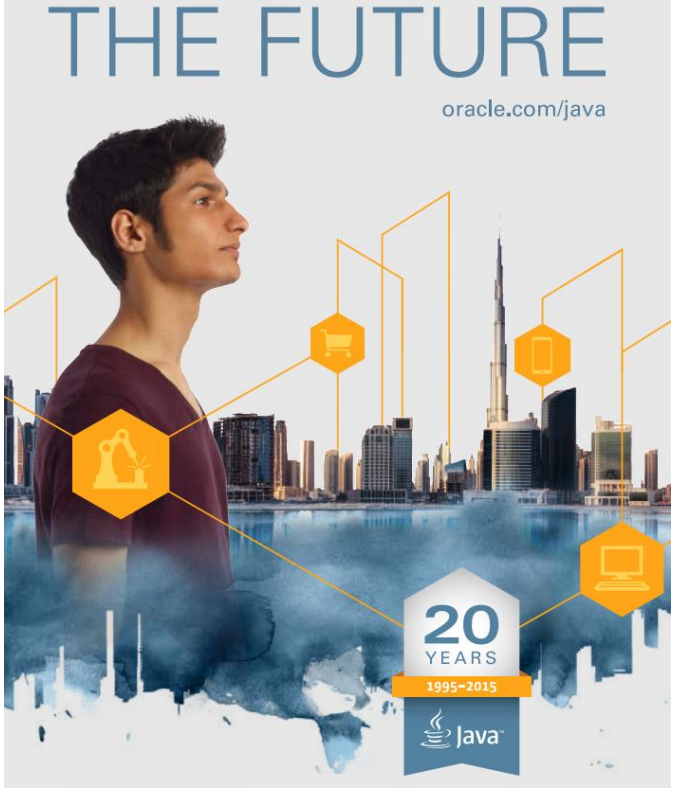
siempre para apoyarte

Visión

Poder aportar al desarrollo del País usando tecnología Java.

CJAVA
siempre para apoyarte

Quienes Somos



Somos una organización peruana orientada a desarrollar, capacitar e investigar tecnología JAVA a través de un prestigioso staff de profesionales a nivel nacional.

Introducción a

Spring Framework 5

Edwin Maraví

emaravi@cjavaperu.com

CJAVA
siempre para apoyarte

¿Qué es Spring?

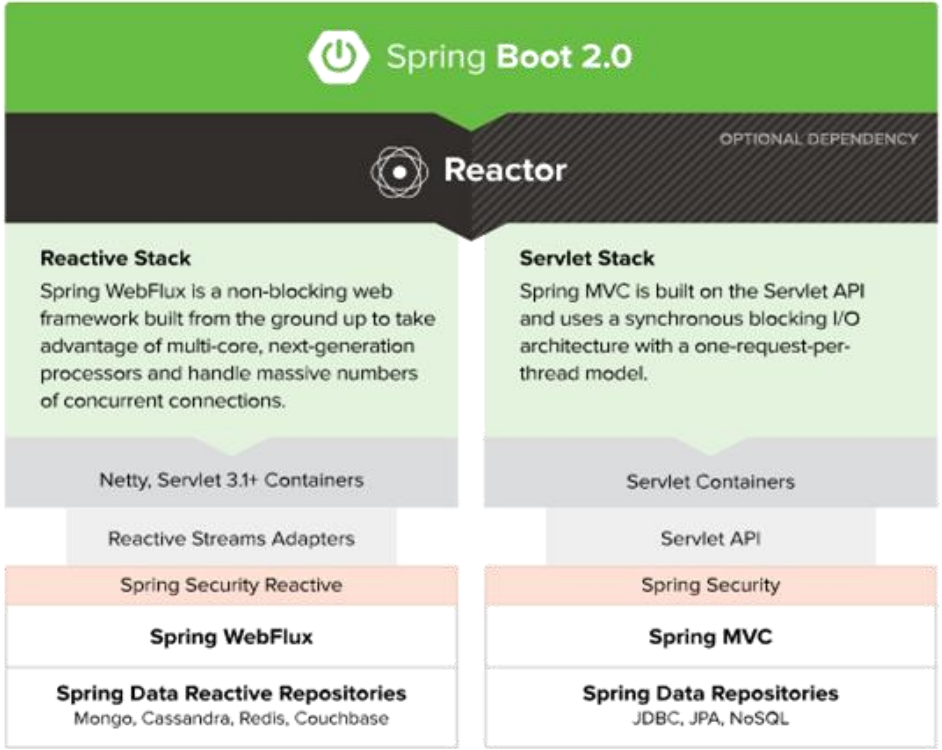
- Spring es un Framework *no-intrusivo* creado por Rod Johnson y descrito en su libro Expert One-on-One J2EE Design and Development
- Basado en el principio de Inversión de Control (IoC).
- Desarrollo a través de Beans y POJOs.
- Integración con otros framework como Struts, Ibatis, Hibernate, Junit, Trapestry, FreeMarker, etc.
- Abstracción para el manejo de Transacciones.
- Soporte para Aspectos (AOP).
- Sigue el paradigma de Convención sobre Configuración (CoC).

CJAVA
siempre para apoyarte

¿Por qué Spring?

- Promueve el desacoplamiento de las capas y la independencia de las mismas.
- Desarrollo orientado a interfaces.
- TDD los POJOs y demás elementos del modelo pueden ser testeadas sin haber definido un framework.
- Promueve el uso de buenas prácticas.
- Se integra con otras tecnologías.
- Es no-intrusivo.
- Posee una arquitectura sólida y extremadamente modular.
- Trae muchas reutilizables y sencillas de heredar.

Spring Framework 5



THE RIGHT TECHNOLOGY STACK FOR THE JOB AT HAND

Developers are constantly challenged with choosing the most effective runtime, programming model and architecture for their application's requirements and team's skill-set. For example, some use cases are best handled by a technology stack based on synchronous blocking I/O architecture while others would be better served by an asynchronous, and non-blocking stack built on the reactive design principles described in the Reactive Streams Specification.

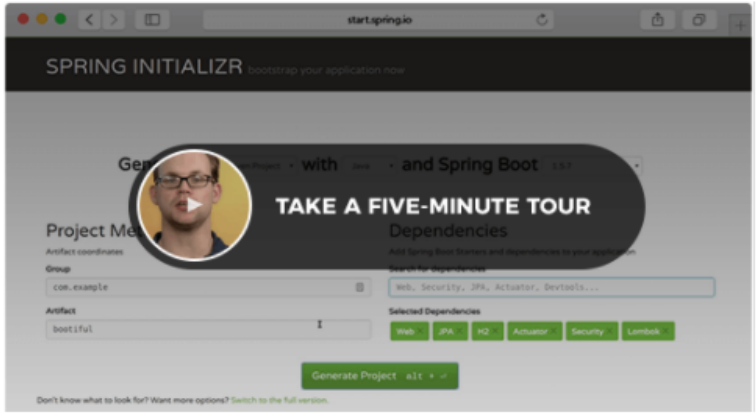
Reactive Spring represents a platform wide initiative to deliver reactive support at every level of the development stack—web, security, data, messaging, etc. Spring Framework 5 delivers on this vision by providing a new reactive web stack called, Spring WebFlux, which is offered side-by-side with the traditional Spring MVC web stack. The choice is yours!

Spring Boot

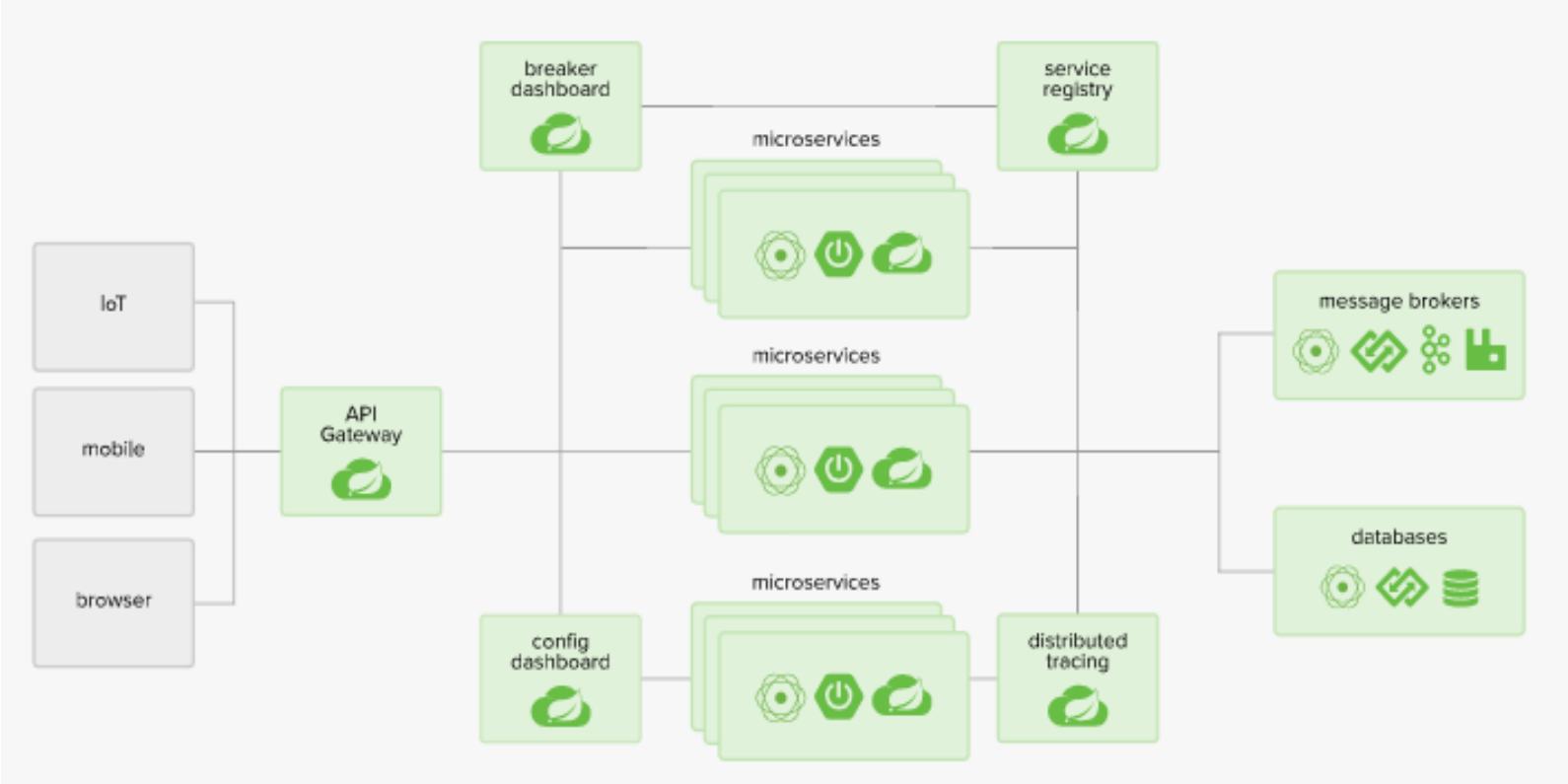
BUILD ANYTHING WITH SPRING BOOT

Spring Boot is the starting point for building all Spring-based applications. Spring Boot is designed to get you up and running as quickly as possible, with minimal upfront configuration of Spring.

- Get started in seconds using Spring Initializr
- Build anything - REST API, WebSocket, Web, Streaming, Tasks, and more
- Simplified Security
- Rich support for SQL and NoSQL
- Embedded runtime support - Tomcat, Jetty, and Undertow
- Developer productivity tools such as live reload and auto restart
- Curated dependencies that just work
- Production-ready features such as tracing, metrics and health status
- Works in your favorite IDE - Spring Tool Suite, IntelliJ IDEA and NetBeans



Spring Cloud

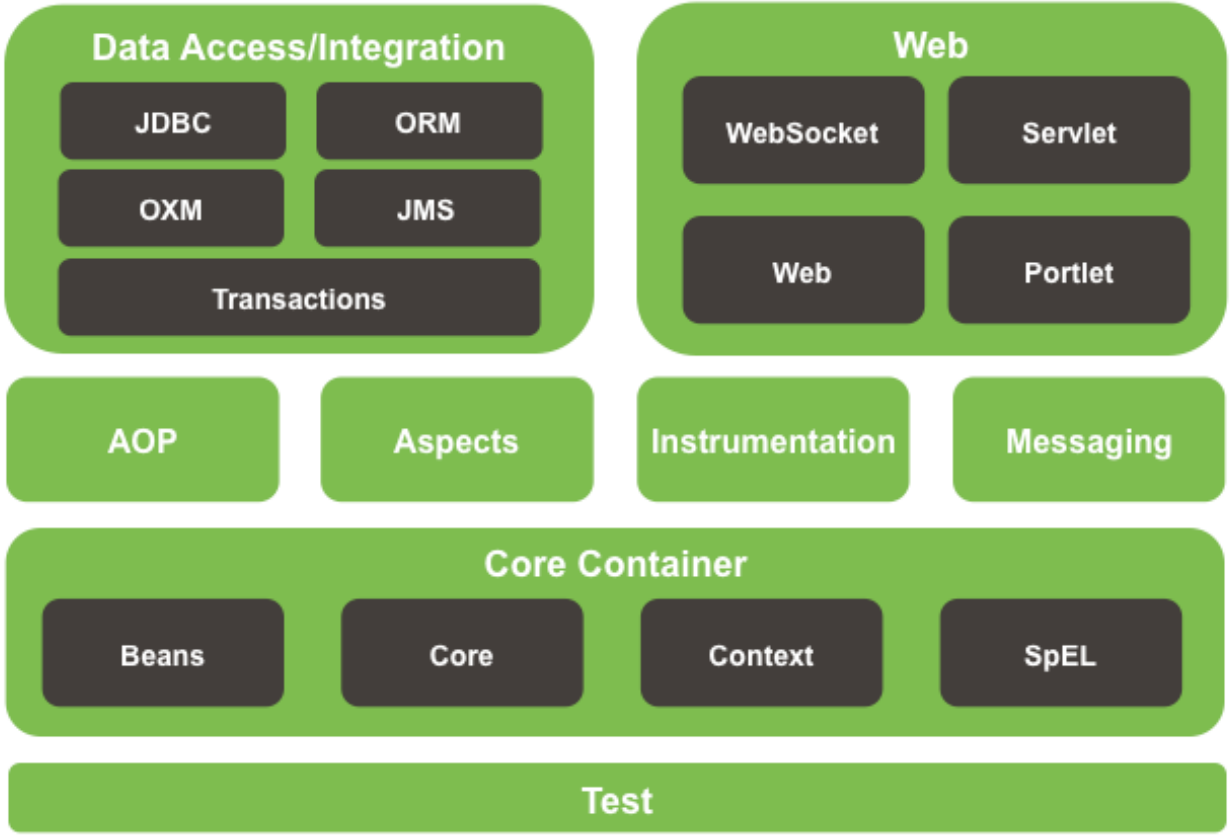


Spring Framework 5

Spring surgió en 2003 como respuesta a la complejidad de las primeras especificaciones J2EE . Mientras que algunos consideran que Java EE y Spring compiten, Spring es, de hecho, complementario a Java EE. El modelo de programación de Spring no incluye la especificación de la plataforma Java EE; más bien, se integra con especificaciones individuales cuidadosamente seleccionadas del paraguas EE:

- Servlet API (JSR 340)
- WebSocket API (JSR 356)
- Utilidades de Concurrencia (JSR 236)
- API JSON Binding (JSR 367)
- Validación de frijol (JSR 303)
- JPA (JSR 338)
- JMS (JSR 914)
- así como las configuraciones de JTA / JCA para la coordinación de transacciones, si es necesario.

Spring Framework Runtime



Inyection of Control IoC

"No nos llames, nosotros te llamaremos" Principio de Hollywood.

- Utiliza la técnica de Inyección de Dependencias o DI, en la cual el contenedor automáticamente asigna (inyecta) implementaciones de objetos cuando y donde se le solicite.

- Formas más comunes de DI:
  - Por constructor: Las dependencias se aplican por medio del constructor de la clase.
  - Por método set: La dependencia es asignada por medio de el método set según la notación JavaBean.

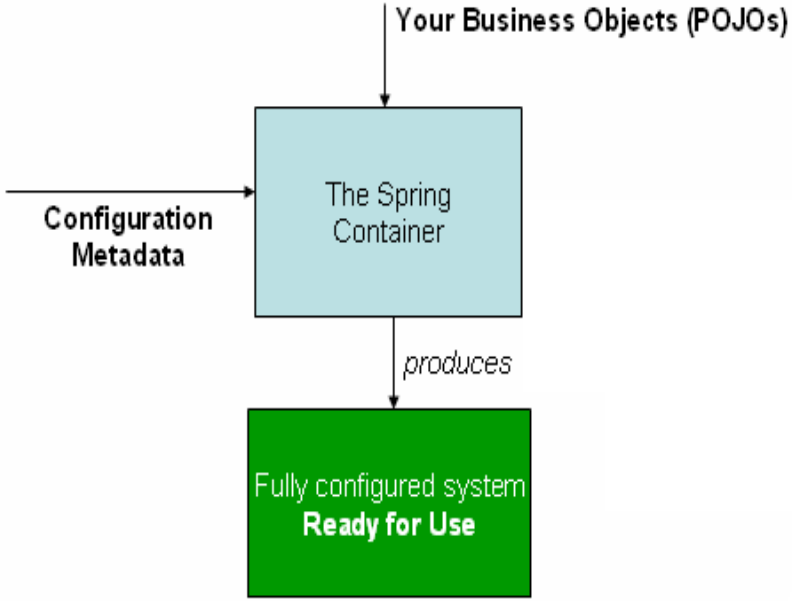


Figura 1. El contenedor Spring IoC

CJAVA
siempre para apoyarte

Spring Framework 5

Refactoring de la Aplicación “Hola Mundo”

Temas

- Como una aplicación tan simple puede ganar agilidad y testeabilidad aplicando conceptos de desacoplamiento con interfaces e Inversión de Control.
- Problemas a solucionar:
  - ¿Como puedo cambiar cierta parte de mi aplicación sin afectar el resto del código?
  - ¿Puedo relacionar diferentes partes de mi aplicación sin tener que escribir mucho código?
  - ¿Cómo puedo probar la lógica de negocio de forma rápida y sencilla sin atarme a un framework en particular?

CJAVA
siempre para apoyarte

Historial de modificaciones

- HolaMundo.
- HolaMundo con argumentos por línea de comandos.
- HolaMundo desacoplado sin interfaces.
- HolaMundo desacoplado usando interfaces.
- HolaMundo desacoplado usando Factorías.
- HolaMundo con usando SpringFramework BeanFactory sin DI.
- HolaMundo con usando SpringFramework con DI.
- HolaMundo con usando SpringFramework con DI y archivos de configuración XML.

1. HolaMundo

HolaMundo

```
//La aplicación HolaMundo solo muestra en consola el mensaje
//"Hola Mundo!"

public class  HolaMundo {
  public static void main(String[] args){
        System.out.println("Hola Mundo!");
  }
}
```

HolaMundo: Identificando Problemas

El código no es extensible, se tendría que cambiar el código (y por tanto recompilar) ante el siguiente escenario:

¿Qué pasaría si quisiera cambiar el mensaje?

HolaMundo: Áreas para refactoring

Crear un mecanismo simple y flexible para cambiar el mensaje

2. HolaMundo con argumentos por línea de comandos

HolaMundo con argumentos por línea de comandos

```
public class  HolaMundo {
  public static void main(String[] args){
        if(args.length > 0){
              System.out.println(args[0]);
        }else {
              System.out.println("Hola Mundo!");
        }
  }
}
```

HolaMundo con argumentos por línea de comandos: Áreas modificadas

Este código externaliza el contenido del mensaje y lo lee en tiempo de ejecución como argumento desde la línea de comandos.

Podemos cambiar el mensaje sin modificar ni compilar el código.

HolaMundo con argumentos por línea de comandos: Identificando Problemas

- El código responsable de mostrar el mensaje (el que llama al método *println* es también el responsable de obtener el mensaje).
  - Cambiar la forma en la que obtenemos el mensaje significaría también cambiar el código que muestra el mensaje.
- El código que muestra el mensaje no puede ser cambiado fácilmente.
  - ¿Qué pasaría si quisiera cambiar la forma en la que se muestra el mensaje, si quisiera mostrar el mensaje como HTML o exportarlo a un archivo de texto?

HolaMundo con argumentos por línea de comandos: Áreas para refactoring

La lógica para mostrar el mensaje debería ser independiente del resto del código.
La lógica para obtener el mensaje debería ser independiente del resto del código,

3. HolaMundo desacoplado sin interfaces

HolaMundo desacoplado sin interfaces

Desacoplar la lógica de la implementación del Proveedor de Mensajes, creando una clase independiente.

```
public class  HolaMundoProveedorDeMensajes {

  public String getMensaje(){
        return "Hola Mundo!";
  }
}
```

HolaMundo desacoplado sin interfaces

Desacoplar la lógica de para mostrar el mensaje del resto del código.

La lógica de mostrado del mensaje recibe un objeto proveedor de mensaje (de cualquier parte), este es un comportamiento de Inyección de Dependencias.

```
public class RenderizadorEstandarDeMensajes{
   private HolaMundoProveedorDeMensajes proveedorDeMensajes = null;
   public String mostrar(){
          if(proveedorDeMensajes == null){
                    throw new RuntimeException("Debe setear un proveesor de mensajes",
                                                  RenderizadorEstandarDeMensajes.class.getName());
          }
          System.out.println(proveedorDeMensajes.getMensaje());
   //continua
```

HolaMundo desacoplado sin interfaces

```
    public void setProveedorDeMensajes(HolaMundoProveedorDeMensajes proveedor) {
      this.proveedorDeMensajes = proveedor;
    }
    public HolaMundoProveedorDeMensajes getProveedorDeMensajes() {
      return this.proveedorDeMensajes;
    }
}
```

HolaMundo desacoplado sin interfaces

```
    //Clase principal
public class HolaMundoDesacoplado{

   public static void main(String[] args){

          RenderizadorEstandarDeMensajes rem = new RenderizadorEstandarDeMensajes();

          HolaMundoProveedorDeMensajes pm = new HolaMundoProveedorDeMensajes();

          rem.setProveedorDeMensajes(pm);

          rem.mostrar();
   }
}
```

HolaMundo desacoplado sin interfaces: Áreas modificadas

La lógica de obtención del mensaje y la lógica encargada de mostrar el mensaje son independientes y están separadas del resto del código.

HolaMundo desacoplado sin interfaces: Identificando Problemas

Tanto la implementación para mostrar el mensaje: `RenderizadorEstandarDeMensajes` como la implementación para obtener el mensaje: `HolaMundoProveedorDeMensajes` están como hard-code en el código principal.

HolaMundo desacoplado sin interfaces: Áreas para refactoring

Hacer que los componentes principales implementen interfaces y que la clase principal use estas interfaces.

4. HolaMundo desacoplado con interfaces

HolaMundo desacoplado con interfaces

La lógica del proveedor de mensajes ahora usa una Interface

```
public interface ProveedorDeMensajes {
    public String getMensaje();
}

public class  HolaMundoProveedorDeMensajes implements ProveedorDeMensajes{

    public String getMensaje(){
          return "Hola Mundo!";
    }
}
```

HolaMundo desacoplado con interfaces

- La lógica para mostrar el mensaje ahora usa una Interface.
- La lógica de mostrado del mensaje recibe un objeto proveedor de mensaje (de cualquier parte), este es un comportamiento de Inyección de Dependencias.

```
public interface RenderizadorDeMensajes {

    public void mostrar();
    public void setProveedorDeMensajes(ProveedorDeMensajes proveedor);
    public ProveedorDeMensajes getProveedorDeMensajes();

}
```

CJAVA
siempre para apoyarte

HolaMundo desacoplado con interfaces

```
public class RenderizadorEstandarDeMensajes implements RenderizadorDeMensajes{

    //ProveedorDeMensajes es ahora una Interface Java
    private ProveedorDeMensajes proveedorDeMensajes = null;

    public String mostrar(){
          if(proveedorDeMensajes == null){
                    throw new RuntimeException("Debe setear un proveesor de mensajes",
                     RenderizadorEstandarDeMensajes.class.getName());
          }
          System.out.println(proveedorDeMensajes.getMensaje());

    //continua
```

HolaMundo desacoplado con interfaces

Desacoplando la lógica de mostrado del mensaje.

```
    public void setProveedorDeMensajes(ProveedorDeMensajes proveedor) {
        this.proveedorDeMensajes = proveedor;
    }
    public ProveedorDeMensajes getProveedorDeMensajes() {
        return this.proveedorDeMensajes;
    }
}
```

HolaMundo desacoplado con interfaces

```
     //Clase principal
public class HolaMundoDesacoplado{

    public static void main(String[] args){

          RenderizadorDeMensajes rm = new RenderizadorEstandarDeMensajes();

          ProveedorDeMensajes pm = new HolaMundoProveedorDeMensajes();

          rm.setProveedorDeMensajes(pm);

          rm.mostrar();
    }
}
```

HolaMundo desacoplado con interfaces: Áreas modificadas

La lógica para mostrar el mensaje puede cambiar sin afectar la forma de obtener el mensaje.

La lógica de obtención del mensaje puede cambiar sin afectar la forma como se muestra el mensaje.

HolaMundo desacoplado con interfaces: Identificando Problemas

Usar una implementación diferente de `RenderizadorDeMensajes` como de `ProveedorDeMensajes` significa cambiar el código y afectar la lógica del negocio, como por ejemplo el código de la clase principal.

HolaMundo desacoplado con interfaces: Áreas para refactoring

Crear una clase Factory que lea los nombres de las implementaciones de nuestras interfaces principales de un archivo *.*properties* y cree una instancia las mismas en tiempo de ejecución.

5. HolaMundo desacoplado usando Factorias

HolaMundo desacoplado usando Factorias

```
public class FabricaDeSoporteDeMensajes {
    private static FabricaDeSoporteDeMensajes instancia = null;
    private Properties props = null;
    private RenderizadorDeMensajes renderer = null;
    private ProveedorDeMensajes proveedor = null;
    private FabricaDeSoporteDeMensajes(){
          props = new Properties();
          try{
          props.load(new FileInputStream("msf.properties"));
          //Obtener el nombre de las Implementaciones
          String clsRender = props.getProperty("renderer.class");
          String clsProveedor = props.getProperty("proveedor.class");
          renderer = (RenderizadorDeMensajes)Class.forName(clsRender).newInstance();
          proveedor = (ProveedorDeMensajes)Class.forName(clsProveedor).newInstance();
          }catch(Exception ex){ ex.printStackTrace();}
    }
```

HolaMundo desacoplado usando Factorias

```
    static {
        instancia = new FabricaDeSoporteDeMensajes();
    }

    public static FabricaDeSoporteDeMensajes getInstancia(){
        return instancia;
    }

    public RenderizadorDeMensajes getRenderizadorDeMensajes(){
        return renderer;
    }

    public ProveedorDeMensajes getProveedorDeMensajes(){
        return proveedor;
    }
}
```

HolaMundo desacoplado usando Factorias

```
public class HolaMundoDesacopladoConFactoria {

    public static void main(String[] args){
        RenderizadorDeMensajes rm = FabricaDeSoporteDeMensajes.getInstancia().
                                    getRenderizadorDeMensajes();

        ProveedorDeMensajes pm = FabricaDeSoporteDeMensajes.getInstancia().
                                    getProveedorDeMensajes();

        rm.setProveedorDeMensajes(pm);

        rm.mostrar();
    }
}
```

HolaMundo desacoplado usando Factorias

Archivo Properties

```
#msf.properties
renderer.class = RenderizadorEstandarDeMensajes
proveedor.class = HolaMundoProveedorDeMensajes
```

HolaMundo desacoplado usando Factorias: Áreas modificadas

Tanto la lógica para mostrar el mensaje como la lógica de obtención del mensaje pueden ser reemplazadas simplemente cambiando el archivo de configuración properties.

- No es necesario un cambio en el código que maneja la lógica de negocio.

HolaMundo desacoplado usando Factorias: Identificando Problemas

Se tiene que escribir mucho código para poder armar toda la aplicación.

- Tuvimos que crear la clase `FabricaDeSoporteDeMensajes`.
- Tenemos que inyectar manualmente una instancia de `ProveedorDeMensajes` en la implementación de `RenderizadorDeMensajes`.

HolaMundo desacoplado usando Factorias: Áreas para refactoring

Podemos usar Spring Framework para manejar los problemas antes mencionados.

Reemplazar la clase `FabricaDeSoporteDeMensajes` por la clase `DefaultListableBeanFactory` de Spring.

Podemos pensar en `DefaultListableBeanFactory` como una versión mas genérica de `FabricaDeSoporteDeMensajes`.

6. HolaMundo usando Spring Framework BeanFactory sin DI

HolaMundo usando Spring Framework BeanFactory sin DI

```
public class HolaMundoSpring {
    public static void main(String[] args) throws Exception{
        //BeanFactory es una clase de Spring
        BeanFactory fabrica = getBeanFactory();
        RenderizadorDeMensajes rm =
        (RenderizadorDeMensajes)fabrica.getBean("renderer");

        ProveedorDeMensajes pm =
        (ProveedorDeMensajes)fabrica.getBean("proveedor");

        rm.setProveedorDeMensajes(pm);
        rm.mostrar();
    }
//continua...
```

HolaMundo usando Spring Framework BeanFactory sin DI

```
//no hay necesidad de entender este codigo
    private static BeanFactory getBeanFactory() throws Exception{
            //obtener la fabrica de Beans
            DefaultListableBeanFactory fabrica = new
                                DefaultListableBeanFactory();
            PropertiesBeanDefinitionReader rdr = new
                                PropertiesBeanDefinitionReader();

            //cargar el archivo de configuracion
            Properties props = new Properties();
            props.load(new FileInputStream("beans.properties"));

            rdr.registerBeanDefinitions(props);

            return fabrica;
    }
}
```

HolaMundo usando Spring Framework BeanFactory sin DI : Áreas modificadas

Quitamos la necesidad de crear una clase propia para la fabrica de beans.

Ganamos una fabrica mucho mas robusta, con un mejor manejo de errores y un mecanismo totalmente desacoplado.

HolaMundo usando Spring Framework BeanFactory sin DI : Identificando Problemas

El código inicial debe tener conocimiento de las dependencias de la clase `RenderizadorDeMensajes`, debe obtener estas dependencias y pasarlas a `RenderizadorDeMensajes`.

- Spring actúa solo como una compleja fábrica de beans, brindando instancias de clase cuando son necesarias.

HolaMundo usando Spring Framework BeanFactory sin DI : Áreas para refactoring

Usar Inyección de Dependencias (DI) de Spring Framework

- Usar la configuración de la BeanFactory de Spring Framework para aplicar la DI.

7. HolaMundo usando Spring Framework con DI

HolaMundo usando Spring Framework con DI

Archivo Properties

```
#Renderizador de Mensajes
renderer.class = RenderizadorEstandarDeMensajes

#Le dice a la clase renderer que necesita de una implementación de
# proveedor mediante la propiedad: proveedorDeMensajes
renderer.proveedorDeMensajes(ref) = proveedor

proveedor.class = HolaMundoProveedorDeMensajes
```

HolaMundo usando Spring Framework con DI

```
public class HolaMundoSpringConDI {
   public static void main(String[] args) throws Exception{

         //BeanFactory es una clase de Spring
         BeanFactory fabrica = getBeanFactory();

         RenderizadorDeMensajes rm =
         (RenderizadorDeMensajes)fabrica.getBean("renderer");

         //ya no se agrega la dependencia del proveedor, porque esta
         // se inyecta automáticamente
         rm.mostrar();
   }
//continua...
```

HolaMundo usando Spring Framework con DI

```
    private static BeanFactory getBeanFactory() throws Exception{
            //obtener la fabrica de Beans
            DefaultListableBeanFactory fabrica = new
                                DefaultListableBeanFactory();
            PropertiesBeanDefinitionReader rdr = new
                                PropertiesBeanDefinitionReader();

            //cargar el archivo de configuracion
            Properties props = new Properties();
            props.load(new FileInputStream("beans.properties"));

            rdr.registerBeanDefinitions(props);

            return fabrica;
    }
}
```

HolaMundo usando Spring Framework con DI

El método main() solo obtiene el bean `RenderizadorDeMensajes` y llama al método mostrar();

- Ya no hay necesidad de agregar manualmente la instancia del proveedor ya que Spring inyecta la dependencia automáticamente.
- A este tipo de comportamiento se le conoce como Inyección de Dependencias.

HolaMundo usando Spring Framework con DI: Observaciones

- No tuvimos que modificar las clases para obtener y mostrar el mensaje.
- Estas clases no tiene ninguna relación ni dependencia con Spring, obvian completamente su existencia.
  - No hubo necesidad de implementar interfaces de Spring
  - No hubo necesidad de extender clases de Spring.
- Estas clases son auténticos POJO's. los cuales pueden ser probados sin necesidad de depender de Spring.

CJAVA
siempre para apoyarte

8. HolaMundo usando Spring Framework con DI y archivos de configuración XML

HolaMundo usando Spring Framework con DI y archivos de configuración XML

Las dependencias de los beans están definidos en archivos XML.

- La configuración basada en archivos XML es mas usada que la configuración basada en archivos properties.

HolaMundo usando Spring Framework con DI y archivos de configuración XML

```
<beans>
   <bean id="renderer" class="RenderizadorEstandarDeMensajes">
        <property name="proveedorDeMensajes">
               <ref local="proveedor">
         </property>
   </bean>

   <bean id="proveedor" class="HolaMundoProveedorDeMensajes">
   </bean>
</beans>
```

HolaMundo usando Spring Framework con DI y archivos de configuración XML

```
public class HolaMundoSpringConDIXML {
    public static void main(String[] args) throws Exception{

        //BeanFactory es una clase de Spring
        BeanFactory fabrica = getBeanFactory();
        RenderizadorDeMensajes rm = (RenderizadorDeMensajes)fabrica.getBean("renderer");
        rm.mostrar();
    }

    private static BeanFactory getBeanFactory() throws Exception{
        //obtener la fabrica
        BeanFactory fabrica = new XmlBeanFactory(new FileSystemResource("beans.xml"));
        return fabrica;
    }
}
```

CJAVA
siempre para apoyarte

```java
ApplicationContext context = new ClassPathXmlApplicationContext("services.xml", "daos.xml");
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- services -->

    <bean id="petStore" class="org.springframework.samples.jpetstore.services.PetStoreServiceImpl">
        <property name="accountDao" ref="accountDao"/>
        <property name="itemDao" ref="itemDao"/>
        <!-- additional collaborators and configuration for this bean go here -->
    </bean>
```

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="accountDao"
        class="org.springframework.samples.jpetstore.dao.jpa.JpaAccountDao">
        <!-- additional collaborators and configuration for this bean go here -->
    </bean>

    <bean id="itemDao" class="org.springframework.samples.jpetstore.dao.jpa.JpaItemDao">
        <!-- additional collaborators and configuration for this bean go here -->
    </bean>

    <!-- more bean definitions for data access objects go here -->

</beans>
```

CJAVA
siempre para apoyarte

```java
// create and configure beans
ApplicationContext context = new ClassPathXmlApplicationContext("services.xml", "daos.xml");

// retrieve configured instance
PetStoreService service = context.getBean("petStore", PetStoreService.class);

// use configured instance
List<String> userList = service.getUsernameList();
```

CJAVA
siempre para apoyarte

IoC: Bean Factory

Un contenedor Spring IoC administra uno o más *beans* . Estos beans se crean con los metadatos de configuración que usted proporciona al contenedor, por ejemplo, en forma de `<bean/>` definiciones XML .

Dentro del contenedor en sí, estas definiciones de bean se representan como `BeanDefinition` objetos, que contienen (entre otra información) los siguientes metadatos:

- *Un nombre de clase calificado para el paquete:* típicamente la clase de implementación real del bean que se está definiendo.
- Elementos de configuración de comportamiento de frijoles, que establecen cómo debe comportarse el frijol en el contenedor (alcance, devoluciones de llamadas del ciclo de vida, etc.).
- Las referencias a otros granos que se necesitan para que el frijol haga su trabajo; estas referencias también se llaman *colaboradores* o *dependencias* .
- Otras configuraciones para establecer en el objeto recién creado, por ejemplo, la cantidad de conexiones que se usarán en un bean que administra un grupo de conexiones, o el límite de tamaño del grupo.

CJAVA
siempre para apoyarte

Bean Naming Conventions

The convention is to use the standard Java convention for instance field names when naming beans. That is, bean names start with a lowercase letter, and are camel-cased from then on. Examples of such names would be (without quotes) `'accountManager'`, `'accountService'`, `'userDao'`, `'loginController'`, and so forth.

Naming beans consistently makes your configuration easier to read and understand, and if you are using Spring AOP it helps a lot when applying advice to a set of beans related by name.

With XML-based configuration metadata you can specify your bean class as follows:

```XML
<bean id="exampleBean" class="examples.ExampleBean"/>

<bean name="anotherExample" class="examples.ExampleBeanTwo"/>
```

CJAVA
siempre para apoyarte

IoC: Dependency Injection

- Se `ApplicationContext` crea e inicializa con metadatos de configuración que describen todos los beans. Los metadatos de configuración se pueden especificar a través de XML, código Java o anotaciones.
- Para cada bean, sus dependencias se expresan en forma de propiedades, argumentos de constructor o argumentos para el método static-factory si está utilizando eso en lugar de un constructor normal. Estas dependencias se proporcionan al bean, *cuando el bean se crea realmente* .
- Cada argumento de propiedad o constructor es una definición real del valor para establecer, o una referencia a otro bean en el contenedor.

```
<bean id="exampleBean" class="examples.ExampleBean">
    <!-- setter injection using the nested ref element -->
    <property name="beanOne">
        <ref bean="anotherExampleBean"/>
    </property>

    <!-- setter injection using the neater ref attribute -->
    <property name="beanTwo" ref="yetAnotherBean"/>
    <property name="integerProperty" value="1"/>
</bean>

<bean id="anotherExampleBean" class="examples.AnotherBean"/>
<bean id="yetAnotherBean" class="examples.YetAnotherBean"/>
```

CJAVA
siempre para apoyarte

```java
public class ExampleBean {

    private AnotherBean beanOne;

    private YetAnotherBean beanTwo;

    private int i;

    public void setBeanOne(AnotherBean beanOne) {
        this.beanOne = beanOne;
    }

    public void setBeanTwo(YetAnotherBean beanTwo) {
        this.beanTwo = beanTwo;
    }

    public void setIntegerProperty(int i) {
        this.i = i;
    }
```

CJAVA
siempre para apoyarte

Links de Referencia

Web Spring

- http://spring.io

Spring Framework

- http://projects.spring.io/spring-framework

Documentación de Referencia

- http://docs.spring.io/spring/docs/4.0.5.RELEASE/spring-framework-reference/htmlsingle

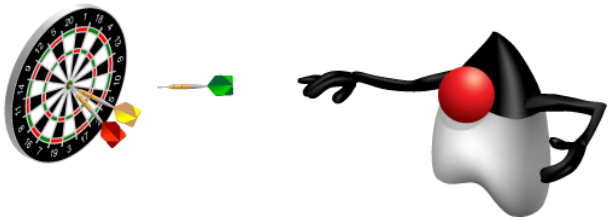
Spring Boot

Edwin Maravi

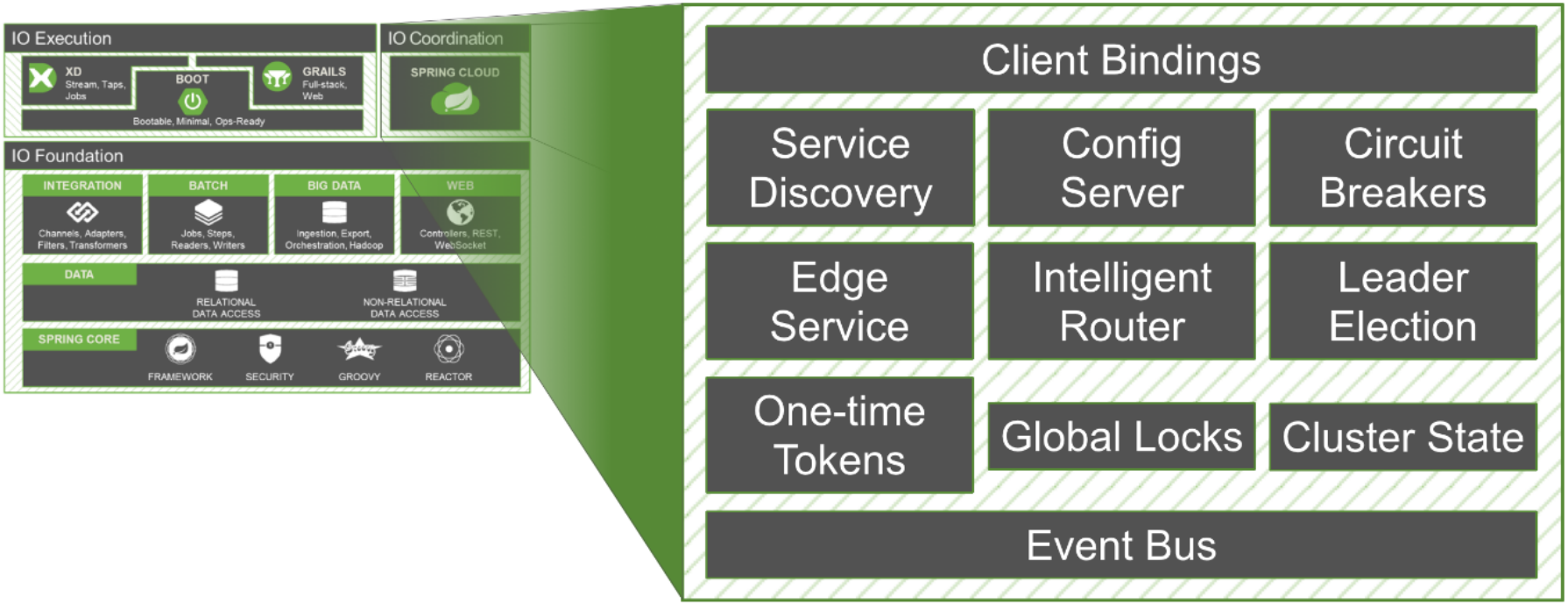
Objetivos

Después de completar esta lección, tu deberías ser capaz de:

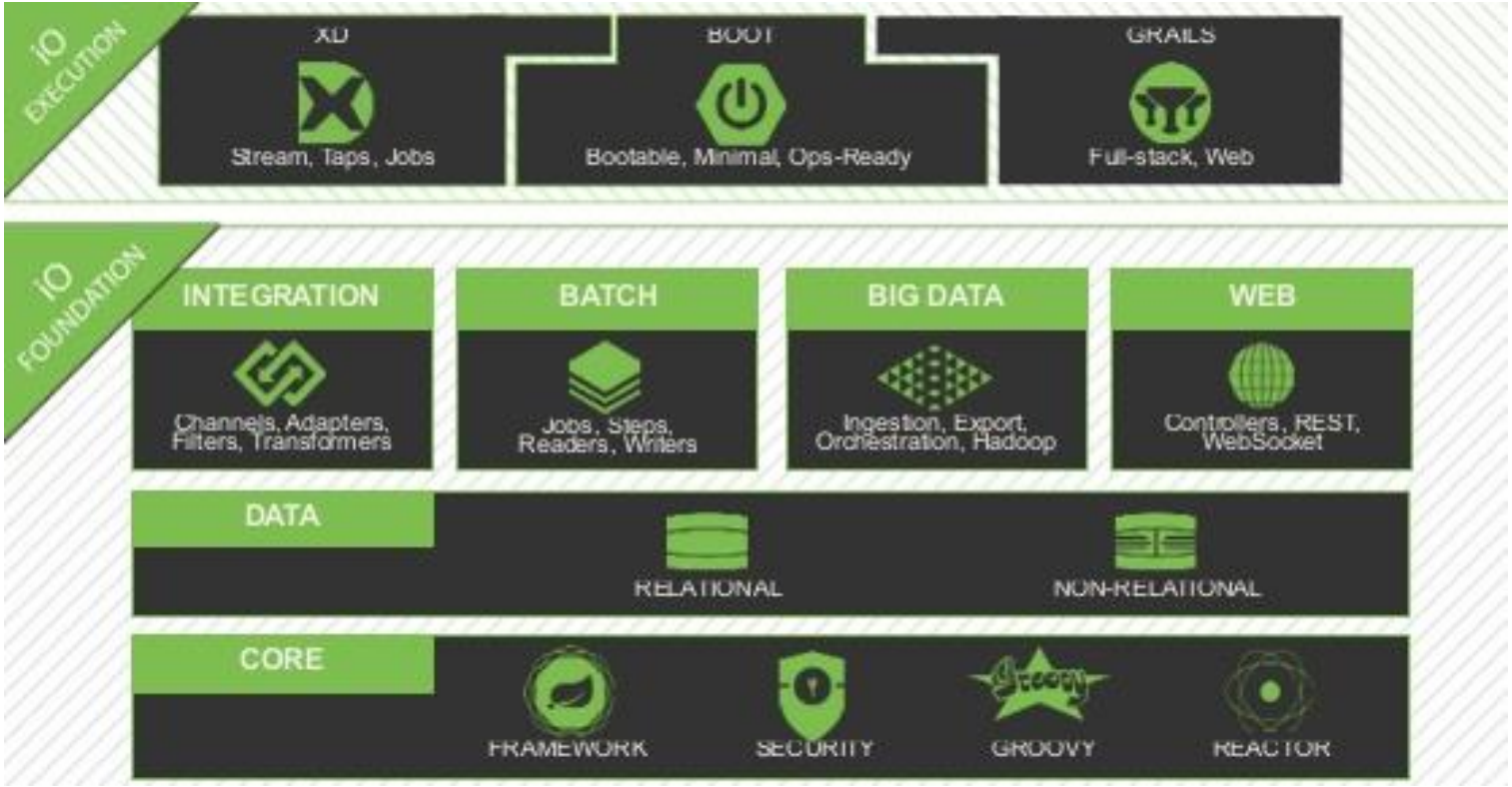
- Construir aplicaciones con Spring Boot
- Enumerar las principales características de Spring Boot
- Conocer las herramientas utilizadas en la actualidad para programación
- Realizar la primera aplicación con Spring Boot



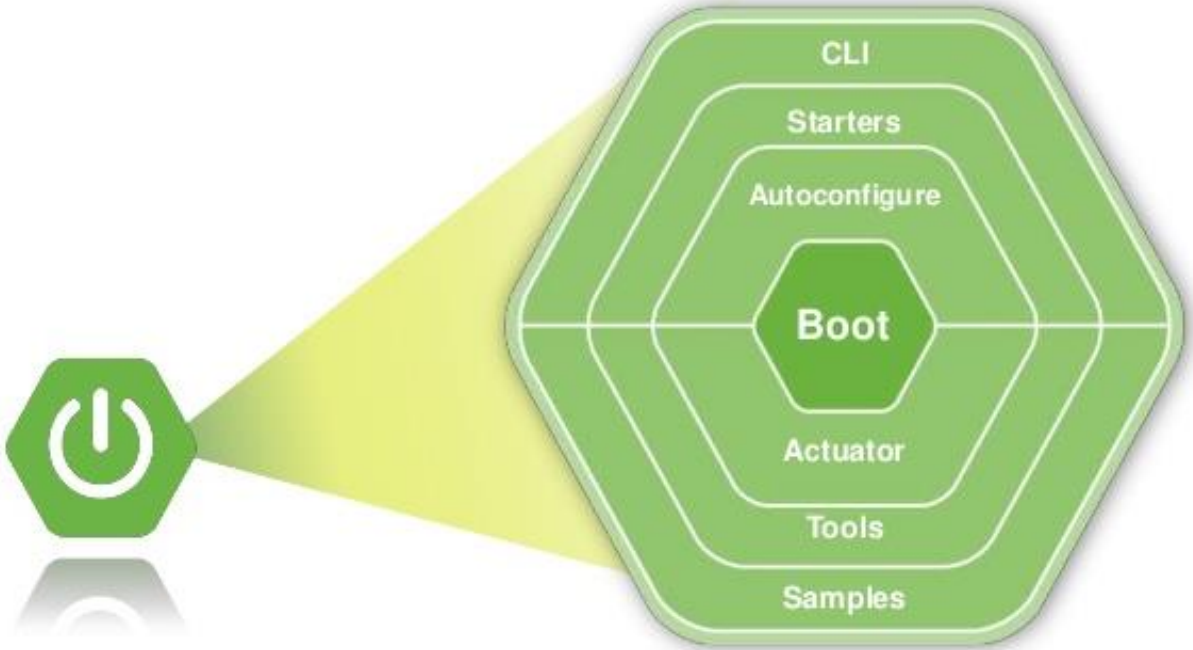
Componentes Spring Cloud



Donde esta Spring Boot



Que es spring boot



¿Qué es Spring Boot?

Al construir aplicaciones con Spring Framework se necesita realizar tres pasos fundamentales.

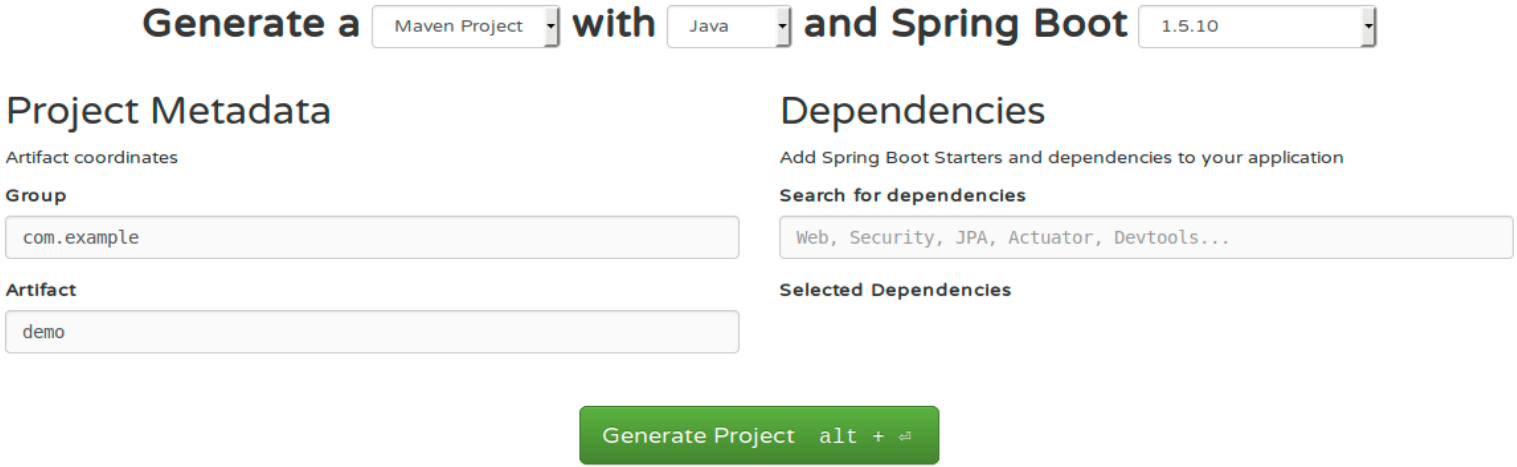
1. Seleccionar jars con maven
2. Crear la aplicación
3. Desplegar en servidor

SpringBoot nace con la intención de simplificar los pasos 1 y 3 y nos centremos en el desarrollo de nuestra aplicación.

¿Qué es Spring Boot?

Spring Boot permite crear standalone, aplicaciones basadas en Spring de tal manera que solo tienes que correr.

Muchas de las aplicaciones de Spring Boot necesitan muy poca configuración de Spring. Spring generator: http://start.spring.io/



Principales características

- Crea aplicaciones Spring standalone.
- Tomcat, Jetty o Undertow embebido directamente (no se necesita deplegar WAR)
- Provee starter POMs para simplificar la configuración Maven
- Configura Spring donde sea posible.
- Provee funciones listas para producción como métricas, health checks y configuración externalizada.
- Sin generación de código y ningún requerimiento de configuración XML.

Herramientas para empezar un proyecto

- Algún IDE como Eclipse o Intelij IDea

- Cualquier motor de base de datos relacional como MySQL, MariaDB, Postgres, SQL Server, Oracle, etc.

- Maven o gradle

Estructura de un archivo POM

```
<parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>1.5.1.RELEASE</version>
</parent>
<properties>
    <java.version>1.8</java.version>
</properties>
<dependencies>
   <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
   </dependency>
</dependencies>
<build>
   <plugins>
      <!-- Package as an executable jar/war -->
      <plugin>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
   </plugins>
</build>
```

Clase inicial

```
@SpringBootApplication
public class SpringBootFirstApplication {

    public static void main(String[] args) {
        SpringApplication application = new SpringApplication(SpringBootFirstApplication.class);
        application.run();

    }
}
```

Mi primera aplicación

CJAVA
siempre para apoyarte

Quiz

¿Cuál es la versión estable de Spring Boot actualmente?

a. 2.0.0.RC2
b. 2.0.0
c. 1.5.10
d. 1.5.11

CJAVA
siempre para apoyarte

Quiz

¿Cómo se llama al archivo de propiedades de Spring boot?

a. springboot.properties
b. spring.properties
c. app.properties
d. application.properties

Principales anotaciones

- @SpringBootApplication
- @ComponentScan
- @EntityScan
- @JpaRepositories
- @EnableAutoConfiguration
- @Autowired, @Controller, @Service, @Repository, @RequestMapping

CJAVA

siempre para apoyarte

Gracias